

Open-Set Fabric Defect Detection With Defect Generation and Transfer

Can Gao^{1b}, *Member, IEEE*, Xiujuan Chen^{1b}, Jie Zhou^{1b}, *Member, IEEE*, Jinbao Wang^{1b}, *Member, IEEE*,
and Linlin Shen^{1b}, *Senior Member, IEEE*

Abstract—Fabric defect detection is indispensable but challenging due to the diversity of fabric texture and defect types in textile mills, and a variety of deep learning-based supervised methods have been introduced to defect detection and segmentation. However, these methods suffer from the scarcity of defective samples and are ineffective in dealing with unknown types of defects. In this study, a novel open-set fabric defect detection method is proposed. To alleviate the problem of scarcity of defective samples, a defect generation and transfer (DGT) module is first designed to generate a large number of defective samples with different defect shapes and types. Then, a defect detection module is constructed, which learns the discriminant representations from real and generated defective samples to identify and segment defects. Finally, an open-set defect classification (DFC) module is introduced, which fits the key features of each known defect type with the Weibull distribution and realizes the classification of known defect types as well as the rejection of unknown defect types. Extensive experiments conducted on five fabric datasets show that the proposed method outperforms other state-of-the-art methods, achieving an average detection performance improvement of 3.0% and 1.4% for image- and pixel-level area under the receiver operating characteristic curve (AUROC), respectively, and an average classification improvement of 8.2% and 17.2% for closed-set and open-set DFC, respectively.

Index Terms—Defect generation, fabric defect detection, mask generator, open-set classification, Weibull distribution.

I. INTRODUCTION

FABRIC is an important and indispensable material in daily life, while fabric inspection is a critical and necessary step in controlling product quality. In most manufacturing factories, fabric inspection generally relies on skilled humans by visual observation. However, prolonged inspection [1] easily causes visual fatigue, thereby leading to a high miss rate. For a long time, the textile industry has been highly

anticipating a machine automation method to replace costly and inefficient manual inspection.

Computer vision-based fabric defect detection [2], [3] is one of the feasible solutions. It captures a series of images from fabrics to be inspected by cameras and uses artificial intelligence methods to detect defective images. Analogous to the inspection process of skilled workers, computer vision-based fabric defect detection can be decomposed into three closely related stages: defect identification, defect segmentation, and DFC. Defect identification refers to determining a fabric image to be defective or normal, whereas defect segmentation aims to locate and segment defects from the identified defective images to aid the assessment of fabric quality. DFC further recognizes the type of defects to accurately assess the quality grade of fabrics because different types of defects have significantly different impacts on quality assessment.

Over the last few decades, many computer vision-based defect detection methods [4] have been proposed for defect identification and defect segmentation. They use handcrafted features to train discriminative or generative models to identify and segment defects from fabric images. However, due to the diversity of defect types and fabric textures, the detection performance based on handcrafted features cannot meet practical needs. Recently, deep learning-based defect identification and segmentation (DIS) methods [5], [6] have attracted considerable attention. These methods are generally categorized into three types: supervised learning, semisupervised learning, and unsupervised learning. Supervised learning-based methods [7] are trained with an amount of labeled data and generally perform better, but the scarcity of defective samples makes it a challenging problem. Semisupervised learning-based methods use a limited amount of labeled data and a large amount of unlabeled data to learn detection models. It typically enlarges the labeled training set by predicting pseudo-labels for unlabeled data. However, mislabeling of unlabeled data can lead to a degradation in model performance, even weaker than the performance that learns on only the limited amount of labeled data. Unsupervised methods [8] are designed to learn the representations of unlabeled data, yet their performance is highly dependent on the data itself because of the lack of external labels.

In contrast, little attention has been paid to DFC [9], but it is a very important problem that deserves to be investigated in depth since the type of defects that occur is a key factor in determining the grade and value of fabrics. DFC can be further classified into closed-set and open-set ones. Closed-set

Received 27 July 2024; revised 31 October 2024; accepted 9 November 2024. Date of publication 4 March 2025; date of current version 20 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476171, Grant 62476172, Grant 62076164, and Grant 62206122; in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515011367 and Grant 2025A1515011511; and in part by Guangdong Provincial Key Laboratory under Grant 2023B1212060076. The Associate Editor coordinating the review process was Dr. Huan Liu. (*Corresponding author: Jie Zhou.*)

Can Gao, Xiujuan Chen, and Linlin Shen are with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with Guangdong Key Laboratory of Intelligent Information Processing, Shenzhen 518060, China (e-mail: 2005gaocan@163.com; cxij@foxmail.com; llshen@szu.edu.cn).

Jie Zhou and Jinbao Wang are with the National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen 518060, China (e-mail: jie_jpu@163.com; wangjb@ieee.org).

Digital Object Identifier 10.1109/TIM.2025.3547485

DFC refers to accurately classifying a defective image into the types of defects seen in the training stage, while open-set DFC aims to not only classify known defects but also identify unknown defects that emerge in the test stage, which is more realistic and practical for real-world application scenarios.

It is known that supervised learning-based methods heavily rely on the quality of labeled data [10], [11]. However, collecting labeled data in practical applications is difficult due to the following main factors: limited defect samples, imbalanced data, and difficulty in labeling. Defect generation [12] may be a promising solution to alleviate the lack of defective samples and the imbalance between different types of defects. Besides, open-set DFC enables the method to detect and classify unknown types of defects. According to the above facts, an open-set fabric defect detection method is proposed, which enriches defect samples through high-quality defect generation. It not only classifies known defect types but also rejects unknown defects. To sum up, the main contributions of this article are summarized as follows.

- 1) To alleviate the scarcity of defective samples, a novel defect generation and transfer (DGT) method is proposed, which could generate defective samples from normal samples and also transfer defects to other samples with different background textures.
- 2) To achieve accurate defect detection, a DIS module is designed, which learns key discriminant features from normal samples, real defective samples, and generated defective samples by the defect generation module with a random mask generator.
- 3) To classify different defects, a DFC module is constructed, which learns the class-specific representation of each known defect type and fits them with the Weibull distribution, thereby achieving accurate classification of known defect types as well as rejection of unknown defect types.

The rest of this article is organized as follows. Section II briefly reviews related studies. Section III describes the details of the proposed method. Section IV reports the experimental results and discusses the performance of the proposed method. In Section V, conclusions are drawn and future work worthy of further research is proposed.

II. RELATED WORK

A. Data Augmentation

Data augmentation aims to enrich the training data by applying various image processing techniques to improve the generalization ability and robustness of the learned model and is mainly categorized into image synthesis and image generation. Image synthesis methods try to synthesize defects and mix them with defect-free images to create defective images. Some methods, such as Cutout [13], Cutmix [14], Perlin-Aug [5], and others [15], [16], [17], [18], have been proposed. However, the output images generated by these methods are primarily unrealistic and have significant differences from the actual defect types. Image generation methods [19] generate new samples by training a generative model to learn the distribution of the data. The methods of multistage GAN [12]

and adversarial transform [20] have been introduced for image generation. However, these generative methods require a large amount of training data, and collecting these data is not practical in reality.

B. Fabric Defect Detection

Defect detection also referred to as anomaly detection is designed to identify defective images and segment abnormal regions. A variety of deep learning-based methods have been proposed and have achieved promising results on different defect detection tasks [21], [22], [23], [24], [25], [26].

For fabric defect detection, some deep learning-based methods have been presented, and these methods can be categorized into supervised, unsupervised, and semisupervised ones in light of whether the training process utilizes label information or not. Supervised methods [2] train models with labeled data, which tend to perform better. These methods achieve desirable results but require a large amount of labeled data to train models. To alleviate this problem, unsupervised methods that rely only on normal samples have been introduced. There are two main methods: reconstruction-based and representation-based. The reconstruction-based method, as detailed in [3] and [27], involves extracting potential features using encoder networks and then reconstructing fabric images with decoder networks. The difference between the original fabric images and the reconstructed ones is used to identify defects. These methods can effectively identify defective samples that are significantly different from normal samples, but they require high quality and quantity of training data and may not be effective in dealing with complex defects. Representation-based methods [6], [28] extract meaningful low-dimensional representations of normal samples, which capture the intrinsic structure and patterns of the data and thus can be used for defect detection. They can automatically learn hierarchical feature representations of data to improve detection accuracy but are sensitive to background noise. Semisupervised methods [29], [30], on the other hand, take advantage of both supervised and unsupervised methods by training with a limited amount of labeled data and a large amount of unlabeled data. Semisupervised fabric defect detection methods improve the generalization ability by utilizing unlabeled data, but the trained model may predict incorrect labels for some unlabeled samples, and learning from these mislabeled data may potentially degrade model performance.

Although existing methods have achieved promising results, their performance still needs to be improved due to the diversity of defects and fabric textures. In addition, these methods mainly focus on the tasks of defect localization. The DFC [31], [32], [33], [34] has not been fully explored, especially the open-set DFC, which is crucial for determining the quality of fabrics.

III. PROPOSED METHOD

Fig. 1 illustrates the overall framework of our method, with three carefully designed modules: DGT, DIS, and DFC. Specifically, the DGT is first used to generate and transfer defects in defect-free images with different fabric textures.

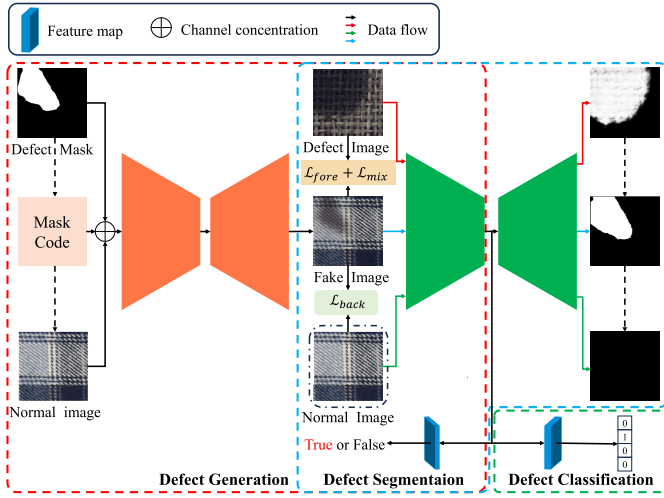


Fig. 1. Pipeline of the proposed method. Defect generation: generating and transferring defects on different textured fabrics. Defect segmentation: identifying and segmenting defects using data enhancement. DFC: classifying the type of defective samples.

Then, the DIS performs DIS with data augmentation by using the pretrained DGT. Finally, the DFC is utilized to achieve DFC for identified defective samples. In the following, each module is described in detail.

A. Defect Generation and Transfer

Given a defective image, previous methods typically label the defect region with only one defect mask, and ultimately, a limited set of training samples is obtained due to the scarcity of collected defective images. To enlarge and diversify fabric defective images, the DGT module is developed to generate defective images on defect-free images with the same fabric texture and transfer defects to defect-free images with different fabric textures.

Specifically, to facilitate the generation of defective images, an incomplete labeling strategy is employed to generate diverse defect masks [see Fig. 2]. Given an original defective image $X^i \in \mathbb{R}^{H \times W \times 3}$ [see Fig. 2(a)], a complete defect mask [Fig. 2(b)] and multiple incomplete defect masks [Fig. 2(c)] are first labeled. Then, the sliding window technique with a window size of $H \times W$ is used to generate complete defect masks $M_{all}^i \in \mathbb{R}^{H \times W}$ [red boxes in Fig. 2(b)] and incomplete defect masks $M^i \in \mathbb{R}^{H \times W}$ [blue boxes in Fig. 2(c)]. Consequently, for each given type of defect, a certain number of defective images $D^i \in \mathbb{R}^{H \times W \times 3}$ can be obtained, with different defect locations and shapes. In addition, a large number of defect-free images $N^i \in \mathbb{R}^{H \times W \times 3}$ [yellow boxes in Fig. 2(a)] can be cropped from normal regions of the original defective image or other normal textured images.

With the defective and defect-free images generated by the incomplete labeling technique, the DGT can be trained to generate defective images and transfer defects to other fabric images with different textures. As shown in Fig. 3, the DGT is mainly composed of data preprocessing, generator ($\mathcal{G}$), and discriminator ($\mathcal{D}$). Specifically, given a defective image D^i with a defect mask M^i and a defect-free image N^i from other types of texture fabrics, a defect code $E_{fore}^i \in \mathbb{R}^{H \times W \times 3}$ and a

normal code $E_{back}^i \in \mathbb{R}^{H \times W}$ are first constructed to represent the foreground defect and background texture information for highly realistic image generation.

To reflect the types of defects and promote the diversity of generated defective images, the foreground defect code $E_{fore}^i \in \mathbb{R}^{H \times W \times S}$ is composed of multiple matrices with binary codes, each of which is constructed by repeating a bit value in the label vector [35] $y^i \in \mathbb{R}^{1 \times S}$ and expanding it to a 2-D matrix with the size of $H \times W$, where $S = \lceil \sqrt{t} \rceil$, t denotes the number of defect types, and $\lceil \cdot \rceil$ indicates upward rounding. In contrast, the background normal code $E_{back}^i \in \mathbb{R}^{H \times W \times 1}$ is set to a matrix with all values equal to 1.

To further realize defect transfer, the defect location and shape information within the defect mask M^i are introduced to the foreground defect code and the background normal code, respectively,

$$\hat{E}_{fore}^i = M^i \otimes E_{fore}^i \quad (1)$$

$$\hat{E}_{back}^i = M^i \otimes E_{back}^i \quad (2)$$

where $\otimes$ denotes the Hadamard product.

In addition, to achieve natural and smooth transitions between foreground defect and background texture in generating defective images, stochastic Gaussian filtering is introduced

$$\tilde{E}_{fore}^i = w_d (G(k_d) \odot \hat{E}_{fore}^i) \quad (3)$$

$$\tilde{E}_{back}^i = w_n (G(k_n) \odot \hat{E}_{back}^i) \quad (4)$$

where $\odot$ denotes the convolution operation; $G(k_d)$ and $G(k_n)$ denote Gaussian filters with the kernel sizes k_d and k_n for foreground defect and background texture, respectively; and w_d and w_n represent the randomized weight parameters, respectively.

Finally, the foreground defect code $\tilde{E}_{fore}^i$, the background normal code $\tilde{E}_{back}^i$, and the defect-free image N^i from other types of texture fabrics are concatenated along the channel dimension to construct the input $\hat{X}^i \in \mathbb{R}^{H \times W \times (S+1+3)}$ of the generator $\mathcal{G}$.

To generate defective images, the generator $\mathcal{G}$ is designed as a decoder and encoder structure, where the encoder is employed to extract features containing information about the background texture of defect-free images and the foreground defect type, location, and shape. While the decoder is to generate the defective image $X_{fake}^i \in \mathbb{R}^{H \times W \times 3}$ from the encoded features. In addition, skip connections are implemented between the layers of the encoder and decoder to preserve both semantic and detailed information about the background texture. To optimize the generated defective images, the defect foreground, texture background, and fusion losses are, respectively, constructed as

$$\mathcal{L}_{fore}^{con} = \|X_{fake}^i - D^i \otimes \tilde{E}_{fore}^i\|_1 \quad (5)$$

$$\mathcal{L}_{back}^{con} = \|X_{fake}^i - N^i \otimes (\mathbf{1} - \tilde{E}_{fore}^i)\|_1 \quad (6)$$

$$\mathcal{L}_{mix} = \|X_{fake}^i - ((D^i \otimes \tilde{E}_{fore}^i) + N^i \otimes (\mathbf{1} - \tilde{E}_{fore}^i)) \otimes (\mathbf{1} - \tilde{E}_{back}^i) + N^i \otimes \tilde{E}_{back}^i\|_1 \quad (7)$$

where $\|\cdot\|_1$ denotes the L_1 loss, $\mathbf{1}$ represents all-one matrix with the same shape as $\tilde{E}_{fore}^i$, X_{fake}^i represents the generated defective image, and D^i stands for the true defective image.

The L_1 loss can effectively evaluate the pixel-value differences between images but does not accurately reflect semantic and structural differences between images. To further optimize the quality of generation, the segmentation losses of the generated defective images, the real defective images, and the real texture background in the defect segmentation module, i.e., $\mathcal{L}_{\text{fake}}^{\text{seg}}$, $\mathcal{L}_{\text{real_fore}}^{\text{seg}}$, and $\mathcal{L}_{\text{real_back}}^{\text{seg}}$, as well as the classification losses of the generated defective images and real defective image in the DFC module, i.e., $\mathcal{L}_{\text{fake}}^{\text{cls}}$ and $\mathcal{L}_{\text{real}}^{\text{cls}}$, are all added, which are detailed in Section III-C. Finally, the generator is optimized by the loss

$$\mathcal{L}_G = \alpha_1 \mathcal{L}_{\text{fore}}^{\text{con}} + \alpha_2 \mathcal{L}_{\text{back}}^{\text{con}} + \alpha_3 \mathcal{L}_{\text{mix}} + \alpha_4 \mathcal{L}_{\text{fake}}^{\text{seg}} + \alpha_5 \mathcal{L}_{\text{real_fore}}^{\text{seg}} + \alpha_6 \mathcal{L}_{\text{real_back}}^{\text{seg}} + \alpha_7 \mathcal{L}_{\text{fake}}^{\text{cls}} + \alpha_8 \mathcal{L}_{\text{real}}^{\text{cls}} \quad (8)$$

where α_i denote the hyperparameter.

To further improve the quality of the generated defective images, a discriminator $\mathcal{D}$ is designed to assist the generator $\mathcal{G}$ and optimized by the loss

$$\mathcal{L}_D = \beta_1 \mathcal{L}_{\text{fore}}^{\text{adv}} + \beta_2 \mathcal{L}_{\text{back}}^{\text{adv}} + \beta_3 \mathcal{L}_{\text{fore}}^{\text{pen}} + \beta_4 \mathcal{L}_{\text{back}}^{\text{pen}} \quad (9)$$

where β_i denotes the hyperparameter and $\mathcal{L}^{\text{adv}}$ and $\mathcal{L}^{\text{pen}}$ represent the adversarial loss and gradient penalty loss to improve generation quality of the generator and the training stability of the discriminator, respectively, which are defined as

$$\mathcal{L}_{\text{fore}}^{\text{adv}} = \mathbb{E}_{D^i \sim p_{\text{data}}(D^i)} [\log(\mathcal{D}(D^i))] + \mathbb{E}_{X_{\text{fake}}^i} [\log(\mathbf{1} - \mathcal{D}(X_{\text{fake}}^i))] \quad (10)$$

$$\mathcal{L}_{\text{back}}^{\text{adv}} = \mathbb{E}_{N^i \sim p_{\text{data}}(N^i)} [\log(\mathcal{D}(N^i))] + \mathbb{E}_{X_{\text{fake}}^i} [\log(\mathbf{1} - \mathcal{D}(X_{\text{fake}}^i))] \quad (11)$$

$$\mathcal{L}_{\text{fore}}^{\text{pen}} = -\mathbb{E}_{\hat{X}_{\text{fore}}^i} \left[\left(\left\| \nabla_{\hat{X}_{\text{fore}}^i} \mathcal{D}(\hat{X}_{\text{fore}}^i) \right\|_2 - \mathbf{1} \right)^2 \right] \quad (12)$$

$$\mathcal{L}_{\text{back}}^{\text{pen}} = -\mathbb{E}_{\hat{X}_{\text{back}}^i} \left[\left(\left\| \nabla_{\hat{X}_{\text{back}}^i} \mathcal{D}(\hat{X}_{\text{back}}^i) \right\|_2 - \mathbf{1} \right)^2 \right] \quad (13)$$

where $\hat{X}_{\text{fore}}^i$ and $\hat{X}_{\text{back}}^i$ denote the images by fusing X_{fake}^i with D^i and N^i , respectively, and $\log(\cdot)$ denotes the logarithmic function.

After the above stage, DGT can generate defect samples with specific shapes based on the input mask. Furthermore, a mask generation module (MGE) is designed to diversify the generated defect samples, as shown in Fig. 4. Specifically, it first initialized a random Gaussian noise vector $Z^i \in \mathbb{R}^{1 \times 2048}$ and the label vector $\mathbf{y}^i \in \mathbb{R}^{1 \times S}$ that indicates the defect type to be generated. Then, the two vectors are concatenated and fed into the generator to generate a defect mask $M_{\text{fake}}^i \in \mathbb{R}^{H \times W \times 1}$ by performing deconvolution and activation. Finally, the generated defect mask is classified by the discriminator to improve the quality of the generator. In the training stage, random rotations are added to the manually labeled mask M_{real}^i to promote the diversity of the generated mask images, and the MGE is optimized by the conditional adversarial loss

$$\mathcal{L}_M^{\text{adv}} = \mathbb{E}_{M_{\text{real}}^i \sim p_{\text{data}}(M_{\text{real}}^i)} [\log(\mathcal{D}_M(\text{Rot}(M_{\text{real}}^i, \theta)))] + \mathbb{E}_{M_{\text{fake}}^i} [\log(\mathbf{1} - \mathcal{D}_M(M_{\text{fake}}^i))] \quad (14)$$

where Rot denotes the random rotation operation with the random angle $\theta \in [0, 90]$ and $\mathbb{E}$ denotes the expectation.

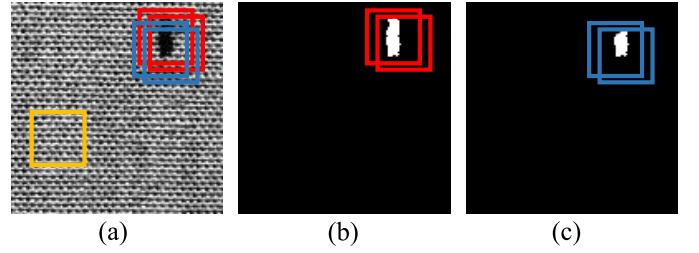


Fig. 2. Generation of defective and defect-free images by incomplete labeling technique. (a) Original defective image. (b) Defect masks with complete labeling. (c) Defect masks with incomplete labeling.

Furthermore, the gradient penalty $\mathcal{L}_M^{\text{pen}}$ is used to enhance the stability of the method training

$$\mathcal{L}_M^{\text{pen}} = -\mathbb{E}_{\hat{M}^i} \left[\left(\left\| \nabla_{\hat{M}^i} \mathcal{D}_M(\hat{M}^i) \right\|_2 - \mathbf{1} \right)^2 \right] \quad (15)$$

where $\hat{M}^i$ denotes the image obtained by uniformly sampling M_{fake}^i and M_{real}^i .

B. Defect Segmentation

With a large number of defective images generated by the trained DGT module, a DIS module shown in Fig. 5 is constructed to identify and segment defects in fabric images. It mainly consists of an encoder and a decoder. First, real defective images, defect-free images, and defective images generated by the DGT are sequentially fed into the encoder to extract rich high-level semantic features, and then, the decoder tries to reconstruct these discriminative features into high-resolution images to realize semantic segmentation. Similar to the DGT, the skip connections are added between different layers of the encoder and decoder to keep more information on the inputs. The DIS loss can be formalized as

$$\mathcal{L}_{\text{real_fore}}^{\text{seg}} = \mathbb{E}_{D^i \sim p_{\text{data}}(D^i)} \left[-\left(M_{\text{all}}^i \log(\text{DIS}(D^i)) + (\mathbf{1} - M_{\text{all}}^i) \log(1 - \text{DIS}(D^i)) \right) \right] \quad (16)$$

$$\mathcal{L}_{\text{real_back}}^{\text{seg}} = \mathbb{E}_{N^i \sim p_{\text{data}}(N^i)} [-\log(\mathbf{1} - \text{DIS}(N^i))] \quad (17)$$

$$\mathcal{L}_{\text{fake}}^{\text{seg}} = \mathbb{E}_{X_{\text{fake}}^i} \left[-\left(M^i \log(\text{DIS}(X_{\text{fake}}^i)) + (\mathbf{1} - M^i) \log(1 - \text{DIS}(X_{\text{fake}}^i)) \right) \right] \quad (18)$$

where $\text{DIS}(\cdot)$ denotes the output of the DIS module.

To enhance the adaption to new types of fabrics, the DIS is further fine-tuned. Specifically, the parameters of the trained DGT are first fixed. Then, the DGT generates defective samples by transferring defects to new types of fabrics. Finally, the DIS is fine-tuned on these defective samples with different fabric textures.

C. Defect Classification

The type of defects that occur is a decisive factor in determining the quality and the grade of fabrics. To realize DFC, a DFC module shown in Fig. 6 is designed. It uses the encoder in the DIS module to extract the key features

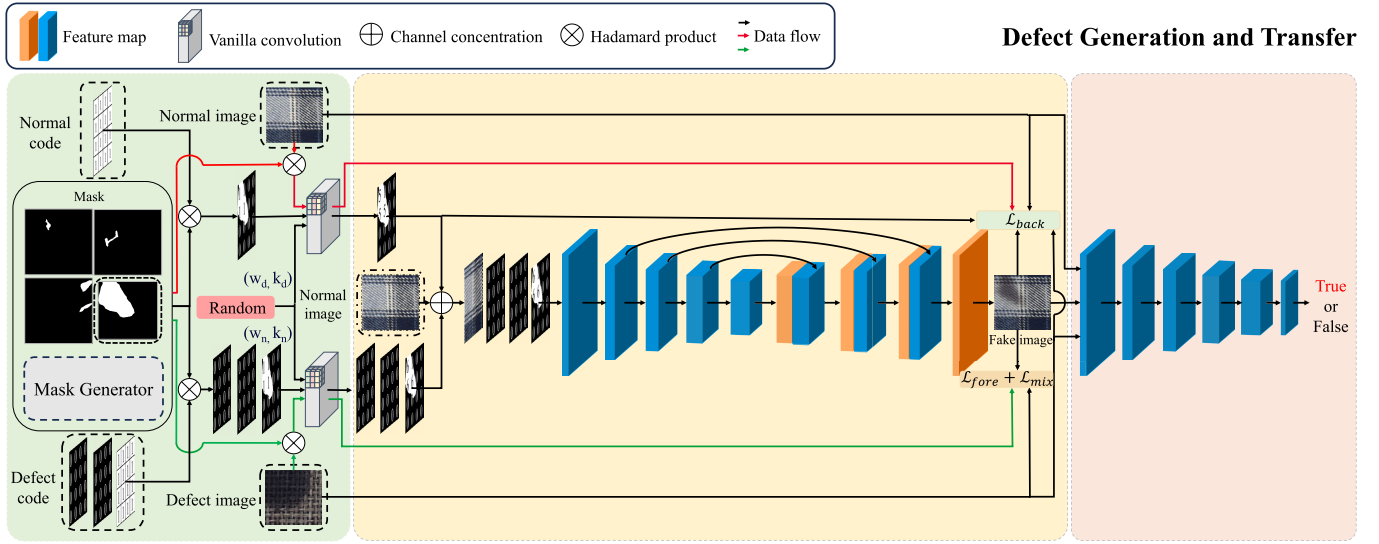


Fig. 3. Flowchart of DGT. It first generates the foreground defect code and the background normal code to reflect the defect and texture information. Then, the normal image along with the foreground and background codes is fed into the generator to generate a fake defective sample. Finally, the discriminator encodes and classifies the fake defective sample to optimize the quality of the generated defective sample.

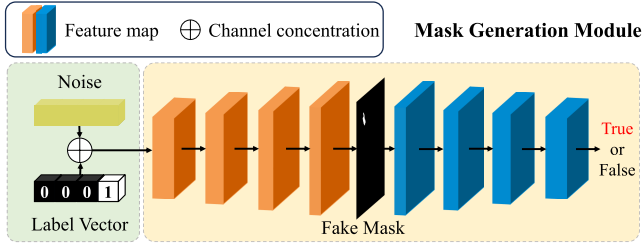


Fig. 4. Flowchart of the MGE. It uses the generator to generate defect masks by feeding the label vector and random noise. Then, the discriminator is employed to encode and classify the real and generated masks to optimize the quality of mask generation.

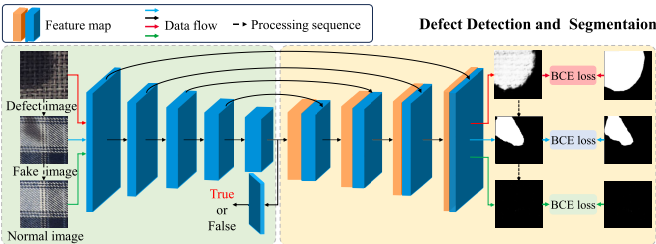


Fig. 5. Flowchart of the DIS module. It first encodes the input images to extract key features. Then, these features are decoded to generate segmentation results.

of defects and further achieves DFC by refining encoded features into a label vector. Specifically, the real defective and defect-free images, as well as the defective images generated by the DGT, are first fed into the encoder of the DIS to extract the discriminative features $F \in \mathbb{R}^{H \times W \times C}$, where C denotes the number of channels. To retain the key information of defects [36], features $\hat{F}_{\text{avg}} \in \mathbb{R}^{1 \times C}$ and $\hat{F}_{\text{max}} \in \mathbb{R}^{1 \times C}$ are obtained by performing average pooling and maximum pooling operations on F in the spatial dimension, respectively. After full convolution operations, the two features are added $\hat{F}_{\text{add}} \in \mathbb{R}^{1 \times C}$. Then, a weight coefficient vector $Q \in \mathbb{R}^{1 \times C}$ is obtained to reflect the importance of each channel in F by mapping the additive features $\hat{F}_{\text{add}}$ with the sigmoid function.

Subsequently, the refined features $\hat{F} \in \mathbb{R}^{H \times W \times C}$ are obtained by weighting the original discriminative features with the learned weight coefficient vector. Finally, these refined features are converted into a label vector by the convolution and mapping operations in the spatial dimension. The DFC module is optimized by the following two losses:

$$\begin{aligned} \mathcal{L}_{\text{real}}^{\text{cls}} &= \mathbb{E}_{D^i \sim p_{\text{data}}(D^i)} \left[-(\mathbf{y}^i \log(\text{DFC}(D^i)) \right. \\ &\quad \left. + (1 - \mathbf{y}^i) \log(1 - \text{DFC}(D^i))) \right] \quad (19) \\ \mathcal{L}_{\text{fake}}^{\text{cls}} &= \mathbb{E}_{X_{\text{fake}}^i} \left[-(\mathbf{y}^i \log(\text{DFC}(X_{\text{fake}}^i)) \right. \\ &\quad \left. + (1 - \mathbf{y}^i) \log(1 - \text{DFC}(X_{\text{fake}}^i))) \right]. \quad (20) \end{aligned}$$

To classify defective images, the DFC is used to extract compact feature representations of different defect types, and the Weibull distributions fit these learned features to classify known classes [32], [37] and reject unknown classes. Specifically, for the c th known defect class, each defective sample is first fed into the DFC to extract key features F_i^c , and then, a class representation features $\bar{F}^c$ is learned by averaging all key features extracted from defective samples within the c th class. To fit the Weibull distribution, the Euclidean distance between the key features F_i^c of each defective sample and the class representation features $\bar{F}^c$ is computed, and some defective samples with maximal distances are used to determine the Weibull distribution

$$\Gamma(\kappa^c, \lambda^c) = \text{Fit} \left(\left\| F_i^c - \bar{F}^c \right\|, \eta \right) \quad (21)$$

where $\text{Fit}(\cdot, \cdot)$ denotes the fitting function of the Weibull distribution; η represents the *tail size*, i.e., the number of selected samples with maximal distances; and $\Gamma(\kappa^c, \lambda^c)$ denotes the fit Weibull distribution with the shape parameter κ^c and the scale parameter λ^c .

To determine the defect type, the defective test sample is first fed into the trained DFC to obtain the key features F_i ,

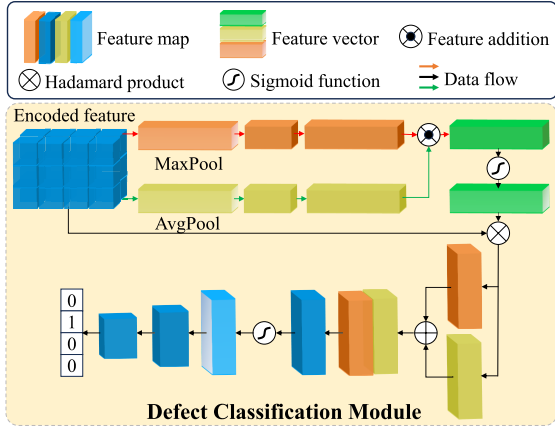


Fig. 6. Flowchart of DFC. It first inputs the encoded features into the DIS encoder. Then, maximum pooling and average pooling operations are performed on the encoded features to obtain the weights of channels. Finally, the weighted features are passed through multiple full convolutional layers to output a one-hot label vector for DFC.

and the belongingness of the test sample to each known class ρ_i^c is computed through the Weibull function $\Gamma(\kappa^c, \lambda^c)$

$$\rho_i^c = 1 - R_{i,\mu}^c \exp\left(-\left(\frac{\|F_i - \bar{F}^c\|}{\lambda^c}\right)^{\kappa^c}\right) \quad (22)$$

where $R_{i,\mu}^c = \max(0, ((\mu - \text{rank}_i^c)/\mu))$ is a heuristic calibrator that assigns larger discounts to more confident classes, μ is set to the number of known classes, and rank_i^c denotes the ranker value of the c th class according to the belongingness of the test samples to all classes.

Subsequently, the AV $\hat{\mathbf{y}}_i = \{\hat{y}_i^1, \hat{y}_i^2, \dots, \hat{y}_i^n\}$ [31] for all known classes is generated and is calibrated by the belongingness to each know classes

$$\tilde{\mathbf{y}}_i = \{\hat{y}_i^1 \rho_i^1, \dots, \hat{y}_i^c \rho_i^c, \dots, \hat{y}_i^n \rho_i^n\}, \quad 1 \leq c \leq n. \quad (23)$$

Accordingly, the possibility that the test sample belongs to unknown classes is computed as

$$\tilde{\mathbf{y}}_i^{n+1} = \sum_{c=1}^n \hat{y}_i^c (1 - \rho_i^c). \quad (24)$$

Finally, the possibility of the test sample to unknown classes $\tilde{\mathbf{y}}_i^{n+1}$ is concatenated to the calibrated AV $\tilde{\mathbf{y}}_i$, and the concatenated vector is mapped into the probabilities to known and unknown classes by the softmax function $\mathbf{y}_i = \text{Softmax}(\tilde{\mathbf{y}}_i, \tilde{\mathbf{y}}_i^{n+1})$. When the probability of the test sample belonging to the unknown class is the largest, the sample is considered to belong to the unknown class; otherwise, it is classified as one of the known classes.

IV. EXPERIMENTS

In this section, a series of experiments is conducted to verify the effectiveness of the proposed method. First, we describe the dataset and experimental setup and then comprehensively evaluate the proposed method in multiple aspects, including DGT, defect detection, and open-set classification. Subsequently, ablation experiments and the inference speed of the

method are discussed, demonstrating its potential for industrial applications.

A. Datasets and Experiment Setups

Experiments involve five datasets: ZJU-Leaper,¹ DAGM,² MVTec,³ SZU-Fabric, and Mix datasets. Table I provides the detailed information on both the training and test sets, and Fig. 7 shows some representative defect samples within these datasets.

- 1) *ZJU-Leaper*: The ZJU-Leaper dataset contains 19 different types of fabric images with a size of 512×512 . For each type of fabric, some common defects such as paint, oil stain, and crimson taint are considered known classes, while other defects are classified as unknown classes.
- 2) *DAGM*: The DAGM dataset has ten different types of fabrics with a size of 512×512 , each of which has different defects that are visually very similar to the background texture. These defects are categorized into linear and nonlinear shapes, where the linear defects, including Classes 2, 4, 6, 8, and 10, serve as known classes, and other nonlinear defects are treated as unknown classes.
- 3) *MVTec*: This dataset has 15 categories with an image size of 1024×1024 . The carpet category is selected in the experiments, containing defects such as color, cut, hole, metal, and thread. Among these defects, color, metal, and thread are classified into known classes, whereas the other defect types are considered unknown classes.
- 4) *SZU-Fabric*: The SZU-Fabric dataset covers three types of fabrics: Patterns 1–3. Each type of fabric contains ten original large-size images of 2456×2058 and 542 small-size images of 512×512 copped from other large-size images. These images contain a total of six types of defects, where thread, metal, crack, slub, and oil stain are categorized as known classes, and button is considered unknown classes.
- 5) *Mix Dataset*: The samples within the Mix dataset are collected from the ZJU-Leaper, DAGM, MVTec, and SZU-Fabric datasets. The Mix dataset involves six common types of defects, including some appearance defects such as oil stain, metal, and paint, and structural defects such as slub, crack, and thread. They are considered known classes, while the other defect types from the four datasets constitute unknown classes.

To enlarge the training set, the incomplete labeling technique is performed on the large-size images of the MVTec and SZU-Fabric datasets. Specifically, one large-size image is selected from each type of defect, and a sliding window with a size of 512×512 and a step of 10 pixels is used to crop 2000 and 2040 defective images for the training set, respectively, with the manual defect masks. For the test set, a sliding window with a size of 512×512 pixels and a large step size

¹<http://www.qaas.zju.edu.cn/zju-leaper/>

²<https://conferences.mpi-inf.mpg.de/dagm/2007/prizes.html>

³<https://www.mvtec.com/company/research/datasets/mvtec-ad>

TABLE I
DETAILED INFORMATION OF THE INVESTIGATED DATASET

Dataset	Stage	Num. of defect types	Num. of defects	Num. of normals	Total
ZJU-Leaper	Train	5	paint(1710), oil stain(1710), crimson taint(1710), cut(\), hole(\), broken end(\), crumple(\), line(\)	5700	10830
	Test	5	paint(1631), oil stain(1534), crimson taint(1747), cut(678), hole(564), broken end(605), crumple(643), line(543)	23704	31649
DAGM	Train	10	Class1(\), Class2(600), Class3(\), Class4(600), Class5(\), Class6(600), Class7(\), Class8(600), Class9(\), Class10(600)	5512	8512
	Test	10	Class1(71), Class2(84), Class3(84), Class4(68), Class5(80), Class6(600), Class7(150), Class8(150), Class9(150), Class10(150)	6996	8050
MVTec	Train	5	color(500), cut(500), thread(1000), hole(\), metal(\)	2000	4000
	Test	5	color(61), cut(33), thread(56), hole(44), metal(50)	70	244
SZU-Fabric	Train	6	thread(425), metal(400), crack(425), slub(400), oil stain(400), button(\)	2100	4140
	Test	6	thread(59), metal(53), crack(50), slub(54), oil stain(56), button(30)	240	542
Mix	Train	9	oil stain(560), metal(560), paint(560), slub(560), crack(560), thread(560), cut(\), button(\), Class9(\)	3360	6720
	Test	9	oil stain(73), metal(68), paint(75), slub(65), crack(79), thread(76), cut(66), button(70), Class9(63)	600	1235

of 150 pixels is employed to crop 314 defective images for testing in the MVTec dataset, while the original 542 small-scale images are directly used to the SZU-Fabric data as the test set. For the ZJU-Leaper and DAGM datasets, considering the small size of images and the complexity of defects, three images of common defect types and seven images of other defects with different textures are selected, and the training set is constructed by manually labeling 30 differently shaped defective regions on each original image to obtain 5130 and 3000 defective images for training, while their test sets use the data defined in the original dataset.

In the experiments, all images are resized to 256×256 , and a batch size of 16 is used. The Adam optimizer with a learning rate of 0.00005 is utilized by the DGT, and the Adam optimizer with a learning rate of 0.0001 is employed for the DIS and the DFC. The exponential decay rates of the Adam optimizer are set to 0.5 and 0.999, respectively. The proposed method is implemented using PyTorch 1.10.0, CUDA 11.3.2, and Python 3.7, and executed on a Ubuntu 20.04 server equipped with a single NVIDIA GPU A100 40 GB.

To evaluate the effectiveness of the proposed method, the AUROC and the per region overlap (PRO) [38] are utilized as image- and pixel-level performance metrics. For open-set recognition, classification accuracy, $F1$ score, and AUROC are used to evaluate closed-set and open-set detection performances, respectively.

B. Performance of DGT

To assess the generated samples, six common defect types are evaluated, including appearance texture defects such as oil stain, metal, and paint, and structural defects such as slub, crack, and thread.

In the proposed method, a mask generator is designed to generate defect masks for diversifying the defective images, and the generated mask images are shown in Fig. 8, where the first row shows the manually labeled masks that can be

used to train the MGE. The remaining rows show the mask images generated from the pretrained MGE.

To examine the capability to generate defect samples, the proposed method is employed to generate common defects such as oil stain, metal, paint, slub, crack, and thread on different fabric textures, and the results are shown in Fig. 9. Moreover, some representative defect generation methods, including Cutout [13], Cutpaste [16], Cutmix [14], SimlexAug [18], and PerlinAug [5], are selected for comparison, and the generated results of all selected methods are shown in Fig. 10. As shown in Fig. 10, the methods of Cutout, Cutpaste, and Cutmix use the cut techniques to generate images, but their synthesized defective images exhibit abrupt edge transitions. Compared to the cut-like methods, SimlexAug and PerlinAug produce better defective images, yet their defect regions seem similar and unrealistic. The proposed method generates defective images using a generative method, which makes the defective regions more visually realistic and the transition between the foreground defects and the background texture much smoother.

To comprehensively assess the quality of generated defect samples, the proposed method and other data augmentation techniques are employed to generate extra training samples to assist the training of the DIS module, and their performance is compared. Specifically, the Cutout, Cutpaste, Cutmix, SimlexAug, and PerlinAug methods are used to generate 1200 defective images for each defect type across various fabrics, and the DIS method is trained on these generated images and evaluated on a test set with 400 different defective images. The DIS performance of the selected methods is shown in Table II.

As shown in Table II, the proposed method performs well in both aspects. For image-level defect identification, the proposed method achieves the best average performance on all defect types with different textures. The performance of the Cutout is mediocre, especially on the fabrics of “Houndstooth”

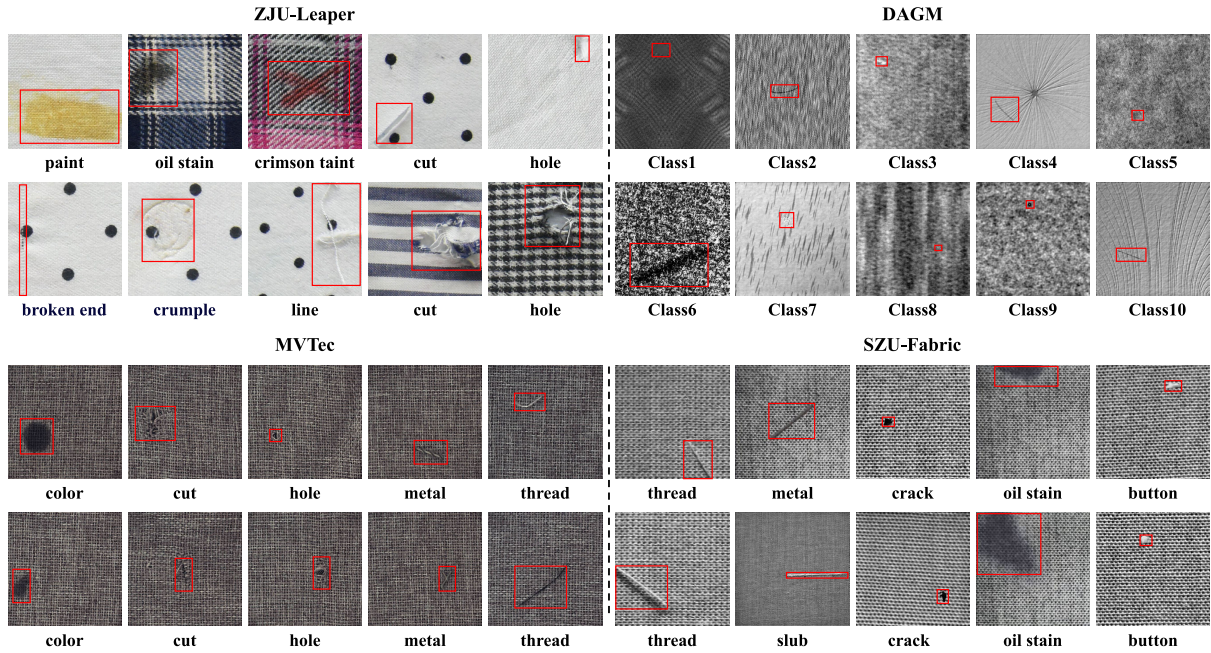


Fig. 7. Representative defect samples within the ZJU-Leaper, DAGM, MVTec, and SZU-Fabric datasets.

and “Knot Pattern.” The performance of the Cutpaste is improved on most textures compared to Cutout, but it performs poorly on “Twill Plaid” and “White Plain.” PerlinAug and SimplexAug simulate defects with noise models, but their performance is still not good on some defect types since the synthesized defects are similar and also different from real defects. Cutmix performs well on appearance defects, with a minimum AUROC of 89.1%, yet its performance on structural defects is mediocre. While DGT gains a minimum AUROC of 93.2% and an average minimum AUROC of 97.8% on all fabrics, which demonstrates that the samples generated by DGT are beneficial for the performance improvement of defect identification. For pixel-level defect segmentation, Cutmix achieves better segmentation results on simple textured fabrics, but poor results on “Houndstooth” and “Knot Pattern.” In contrast, the proposed method achieves an average AUROC of at least 97.5% on all fabrics. This result can be explained by that DGT generates more realistic samples, thus facilitating the segmentation performance of the proposed method.

C. Performance of Defect Detection

To evaluate defect detection performance, the proposed method is compared with other defect detection methods, including four supervised methods Unet [21], SegNet [22], FssNet [23], and CGNet [24], as well as defect detection methods with data augmentation techniques DRÆM [5], Mem-Seg [17], RD [25], RD++ [18], and DeSTSeg [26].

The DIS performance of different methods are shown in Tables III and IV. It is observed that the proposed method outperforms other defect detection methods in all three metrics. For defect identification, the proposed method achieves an average AUROC of 97.6%, which is 3% higher than DeSTSeg. Particularly on the SZU-Fabric dataset, its defect identification performance is close to 100%, indicating the

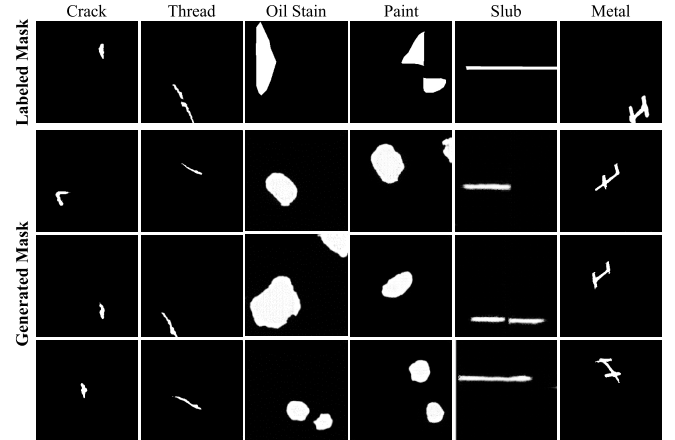


Fig. 8. Defect masks incompletely labeled by manual and automatically generated by the proposed MGE.

good generalization ability of the proposed method. For defect segmentation, the proposed method achieves an average AUROC of 97.6%, which is 1.4% higher than DeSTSeg and 2.3% higher than RD++. In terms of the PRO metric, although the proposed method achieves only 85.3% on average in ZJU-Leaper and DAGM, the proposed method presents a greater superiority with respect to other methods, especially on the SZU-Fabric dataset. This shows that the proposed method achieves effective segmentation in both DIS. Fig. 11 shows the defect segmentation results. Compared with other methods, the proposed method is more precise in segmenting the defect region and also avoids misclassifying defect-free regions as defective, thereby resulting in high defection accuracy and low false alarm rate. This may be attributed to the fact that the DIS module enhances the ability of feature extraction while optimizing the DGT, thus being able to learn a more

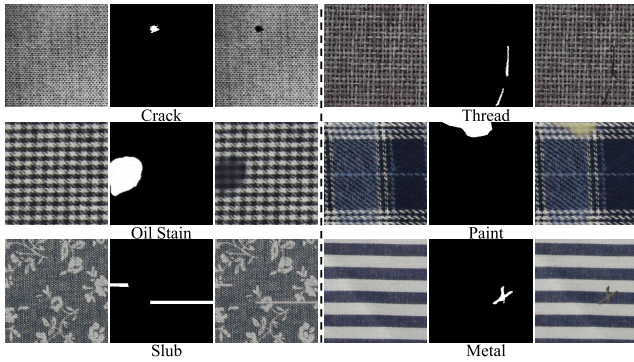


Fig. 9. Defect samples generated by the proposed method on different fabric textures, where the first column indicates the normal fabric image, the second column denotes the defect mask generated by the proposed MGE, and the last column shows the generated defect samples.

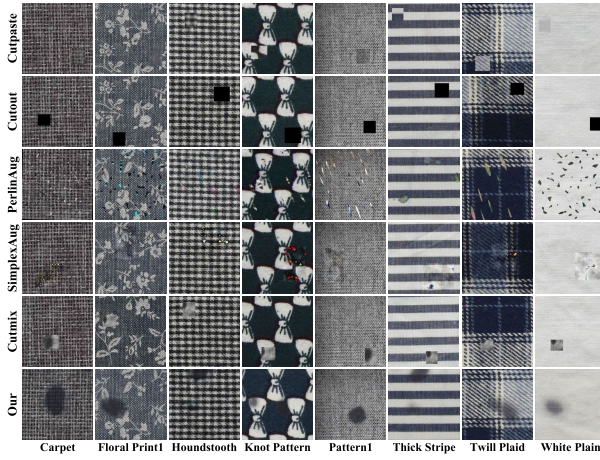


Fig. 10. Defect samples generated by different methods, where each row denotes the results of one selected method on different fabric textures.

favorable feature representation for defect segmentation. These results demonstrate that the proposed method is effective in identifying and segmenting defects from fabric images with different textures.

D. Performance of Open-Set DFC

The proposed method is compared with other state-of-the-art open-set recognition methods, including CROSR [32], Openmax [31], Softmax [39], CSI [40], RPL [41], ARPL [33], and CSSR [34]. Since the unavailability of a large number of real defect samples, 5000 images are generated for each known defect class using the trained DGT module with random defect masks generated by the MGE module. Then, the features extracted by the DFC module are used to fit the Weibull distribution of each known defect class. The “libmr” library [37] is employed to compute the parameters of the Weibull distribution. In the experiments, the *tail size* η that reflects the number of extremes in the tail of the distribution is uniformly set to 5, and the parameter μ that is used to determine the defect class is set to the number of known classes.

Table V shows the closed-set and open-set classification performance of all selected methods. As shown in Table V, the proposed method performs well in both aspects. For closed-set classification, the proposed method achieves an average classification accuracy of 93.9%, which is improved over the

TABLE II
PERFORMANCE OF ALL SELECTED DATA ENHANCEMENT METHODS OVER DIFFERENT FABRICS IN IMAGE-LEVEL AUROC (%) / PIXEL-LEVEL AUROC (%)

	Method	Crack	Thread	Oil Stain	Paint	Slub	Metal	Avg
Pattern1	Cutpaste	89.8 / 91.1	93.5 / 95.1	97.8 / 90.2	94.6 / 96.4	95.4 / 94.5	90.2 / 89.1	93.6 / 92.7
	Cutout	99.5 / 99.3	71.1 / 95.2	90.1 / 76.9	93.9 / 91.3	72.8 / 56.6	88.4 / 73.7	84.1 / 82.2
	PerlinAug	90.2 / 92.8	91.5 / 99.1	98.3 / 95.1	93.5 / 96.2	73.9 / 71.8	91.2 / 89.8	89.8 / 90.8
	SimplexAug	91.3 / 93.7	92.0 / 98.5	90.0 / 93.8	93.7 / 88.8	95.0 / 63.7	80.3 / 87.8	90.4 / 87.7
	Cutmix	98.8 / 96.9	99.5 / 98.2	89.1 / 78.9	95.5 / 86.5	87.9 / 65.5	90.0 / 96.7	93.5 / 87.1
	Our	99.8 / 99.5	100.0 / 99.8	98.5 / 97.2	96.6 / 97.4	100.0 / 99.5	100.0 / 98.9	99.1 / 98.7
Floral Print1	Cutpaste	67.6 / 81.0	75.2 / 93.6	95.6 / 96.2	93.1 / 96.9	74.2 / 82.7	99.2 / 97.3	84.1 / 91.3
	Cutout	99.9 / 98.4	71.8 / 86.6	96.5 / 81.3	91.8 / 86.7	55.4 / 62.1	81.2 / 83.3	82.8 / 83.1
	PerlinAug	54.6 / 68.4	55.3 / 68.1	92.9 / 89.4	74.0 / 65.0	54.8 / 39.6	55.6 / 69.7	64.5 / 66.7
	SimplexAug	97.6 / 34.4	69.4 / 52.5	93.5 / 54.5	94.4 / 85.5	70.0 / 77.7	78.7 / 73.3	83.9 / 63.0
	Cutmix	97.6 / 97.3	65.5 / 96.9	94.4 / 91.7	95.0 / 93.6	81.8 / 71.4	100.0 / 97.6	89.0 / 91.4
	Our	99.9 / 98.9	93.2 / 98.8	97.5 / 97.0	96.0 / 97.2	100.0 / 98.5	100.0 / 98.9	97.8 / 98.2
Carpet	Cutpaste	90.0 / 96.9	91.9 / 54.6	96.9 / 96.5	97.6 / 96.6	92.1 / 41.0	96.0 / 86.4	94.1 / 78.7
	Cutout	99.8 / 97.7	74.4 / 63.5	95.4 / 85.3	89.0 / 81.9	81.9 / 58.8	81.8 / 51.3	87.0 / 73.1
	PerlinAug	53.0 / 95.2	74.7 / 80.5	96.1 / 95.4	96.3 / 96.3	20.5 / 32.6	78.7 / 73.3	70.8 / 82.6
	SimplexAug	97.4 / 97.6	65.2 / 51.0	86.9 / 77.4	95.5 / 91.7	96.8 / 94.5	35.6 / 60.7	79.6 / 78.8
	Cutmix	99.2 / 97.6	93.6 / 60.6	95.6 / 91.8	93.5 / 87.2	89.0 / 79.9	97.3 / 79.9	94.7 / 82.8
	Our	99.8 / 98.5	99.9 / 96.5	99.6 / 97.6	99.7 / 97.7	100.0 / 97.6	100.0 / 98.8	99.8 / 97.8
Knot Pattern	Cutpaste	73.0 / 65.9	74.9 / 63.0	80.2 / 79.2	94.0 / 85.8	75.5 / 50.8	78.3 / 76.0	79.3 / 70.1
	Cutout	99.4 / 95.9	46.0 / 63.3	86.4 / 56.4	92.8 / 66.9	56.6 / 58.8	46.1 / 66.2	71.2 / 67.9
	PerlinAug	57.9 / 79.4	60.7 / 66.6	90.7 / 65.6	93.4 / 93.7	60.5 / 50.3	80.5 / 84.2	74.0 / 73.3
	SimplexAug	32.0 / 64.1	63.0 / 78.0	20.8 / 56.6	87.9 / 87.8	81.0 / 76.2	89.0 / 88.0	53.9 / 75.1
	Cutmix	74.3 / 75.3	79.3 / 85.6	93.0 / 73.0	91.9 / 80.7	86.2 / 86.0	99.9 / 96.8	87.4 / 82.9
	Our	99.6 / 96.7	97.7 / 98.6	96.3 / 97.2	95.4 / 97.4	100.0 / 96.3	100.0 / 98.9	98.2 / 97.5
Houndstooth	Cutpaste	76.0 / 95.5	52.8 / 80.8	57.1 / 48.4	97.2 / 95.4	91.5 / 89.3	98.2 / 98.0	78.8 / 84.6
	Cutout	99.6 / 95.7	55.7 / 51.8	69.5 / 34.4	96.1 / 87.3	62.6 / 57.7	56.5 / 59.6	73.3 / 64.4
	PerlinAug	32.3 / 47.0	37.5 / 63.9	79.6 / 72.9	96.0 / 96.4	67.8 / 71.1	97.9 / 98.7	68.5 / 75.0
	SimplexAug	52.6 / 63.1	47.6 / 44.2	69.2 / 31.7	96.0 / 92.7	79.3 / 76.9	74.0 / 86.7	69.8 / 65.9
	Cutmix	99.5 / 91.3	87.4 / 58.8	93.0 / 78.6	97.6 / 91.1	60.7 / 69.3	100.0 / 97.3	89.7 / 81.1
	Our	99.8 / 98.6	99.1 / 96.5	99.2 / 97.4	98.2 / 97.6	100.0 / 97.6	100.0 / 98.9	99.4 / 97.8
White Plain	Cutpaste	96.8 / 92.9	96.9 / 93.2	95.9 / 96.4	95.5 / 95.9	43.5 / 75.9	91.9 / 90.5	86.8 / 90.8
	Cutout	100.0 / 98.9	91.0 / 89.9	96.9 / 94.6	96.1 / 91.4	95.0 / 95.8	92.3 / 89.0	95.2 / 93.3
	PerlinAug	94.0 / 89.0	92.3 / 91.0	94.9 / 96.9	94.0 / 96.9	91.0 / 90.1	96.7 / 92.4	93.8 / 92.7
	SimplexAug	88.4 / 91.9	95.5 / 91.7	95.2 / 93.7	96.0 / 91.8	64.2 / 89.2	91.5 / 90.9	88.5 / 91.5
	Cutmix	95.5 / 92.9	93.3 / 91.9	96.9 / 95.2	98.4 / 91.5	96.0 / 97.5	92.5 / 95.6	95.4 / 94.1
	Our	100.0 / 99.4	99.6 / 97.7	98.8 / 97.6	100.0 / 97.8	98.0 / 98.6	98.9 / 98.9	99.2 / 98.3
Thick Stripe	Cutpaste	98.3 / 97.4	85.2 / 93.0	96.5 / 96.1	94.5 / 89.1	84.0 / 83.3	97.5 / 93.7	92.7 / 92.1
	Cutout	99.7 / 95.7	93.0 / 95.3	97.3 / 81.0	96.4 / 82.4	66.1 / 74.3	92.0 / 93.9	90.8 / 87.1
	PerlinAug	67.7 / 58.0	68.3 / 58.8	75.4 / 66.3	82.6 / 62.2	68.3 / 54.9	68.6 / 52.2	71.8 / 58.7
	SimplexAug	99.8 / 94.7	81.7 / 98.4	95.5 / 95.5	95.7 / 96.2	72.8 / 84.0	95.5 / 95.4	90.2 / 94.0
	Cutmix	96.0 / 93.3	98.9 / 96.4	96.0 / 91.9	95.9 / 92.9	85.6 / 85.0	96.0 / 94.7	94.7 / 92.4
	Our	99.9 / 98.9	99.1 / 99.2	98.0 / 97.5	97.3 / 97.3	100.0 / 98.5	98.9 / 97.9	98.9 / 98.2
Twill Plaid	Cutpaste	70.2 / 93.4	70.5 / 88.5	96.9 / 95.0	91.0 / 95.8	59.6 / 71.1	96.8 / 94.1	80.8 / 89.6
	Cutout	99.9 / 98.4	62.4 / 66.4	82.2 / 55.0	91.9 / 88.5	60.4 / 63.3	89.7 / 78.4	81.1 / 75.0
	PerlinAug	47.5 / 43.9	48.5 / 48.1	47.9 / 64.2	50.4 / 50.4	66.9 / 33.1	48.2 / 46.8	51.6 / 47.8
	SimplexAug	66.7 / 59.2	66.7 / 64.7	86.8 / 67.5	89.3 / 42.0	86.0 / 42.3	87.0 / 81.2	80.4 / 49.5
	Cutmix	94.2 / 93.0	89.4 / 86.2	92.5 / 85.5	95.3 / 90.8	89.5 / 74.3	94.0 / 94.3	92.5 / 87.4
	Our	99.9 / 99.5	99.6 / 97.7	97.8 / 97.1	98.6 / 97.6	100.0 / 98.3	98.6 / 98.9	99.1 / 98.2

best comparison method Openmax and the second-place comparison methods Softmax by 8.2% and 8.8%, respectively. For open-set classification, the proposed method gains an average AUROC of 97.2%, which is much higher than that of the best comparison method CSSR and the second-place comparison method RPL, with a performance improvement of 17.2% and 19.9%, respectively. In addition, the proposed method also outperforms the other methods in terms of *F1* score, with an average value of 91.4%, which is 3.8% higher than the second-place method Openmax, indicating that the proposed method also obtains a better performance when dealing with the classification imbalance case. A possible explanation for this is that the proposed method focuses more on defect regions, which facilitates the extraction of key features and weakens the negative effect of texture background, resulting in better performance. These results demonstrate that the proposed method can effectively classify known defect types and detect unknown defect types.

E. Ablation Experiments

In this section, ablation experiments are conducted to test the validity of each module as well as the effect of the parameters in the proposed method.

TABLE III
DEFECT IDENTIFICATION PERFORMANCE OF ALL SELECTED METHODS IN
IMAGE-LEVEL AUROC (%)

Method	Unet	SegNet	FssNet	CGNet	DRÆM	MemSeg	RD	RD++	DeSTSeg	Ours
ZJU-Leaper	79.9	83.1	82.2	88.3	76.8	79.1	96.5	97.9	96.8	98.2
DAGM	85.2	84.1	80.5	88.5	80.5	82.1	82.5	89.6	87.1	96.5
MVTec	95.2	89.5	96.9	83.4	97.6	72.3	92.9	87.5	96.6	98.8
SZU-Fabric	98.2	97.6	98.9	86.8	97.4	94.3	99.5	99.2	99.4	99.8
Mix	88.3	80.2	88.1	82.2	93.7	73.4	85.2	87.9	93.1	94.8
Avg	89.4	86.9	89.3	85.8	89.2	80.2	91.3	92.4	94.6	97.6

TABLE IV
DEFECT SEGMENTATION PERFORMANCE OF ALL SELECTED METHODS IN PIXEL-LEVEL AUROC(%)/PRO(%)

Method	Unet	SegNet	FssNet	CGNet	DRÆM	MemSeg	RD	RD++	DeSTSeg	Ours
ZJU-Leaper	85.4 / 54.7	85.2 / 51.1	82.8 / 57.4	74.6 / 54.0	81.3 / 53.0	75.1 / 46.0	94.6 / 80.2	95.7 / 81.5	94.8 / 80.9	97.6 / 85.3
DAGM	86.1 / 57.4	74.0 / 50.8	71.3 / 54.2	87.2 / 50.9	90.1 / 67.4	80.3 / 56.7	97.6 / 81.9	98.2 / 86.0	98.0 / 83.5	98.5 / 86.5
MVTec	94.9 / 90.0	97.2 / 89.3	90.1 / 86.9	91.2 / 74.4	96.3 / 84.6	70.6 / 52.2	96.3 / 91.5	95.6 / 89.7	97.1 / 89.2	98.4 / 94.5
SZU-Fabric	98.7 / 96.3	97.4 / 93.0	97.2 / 90.7	82.9 / 77.6	92.4 / 78.2	81.0 / 72.2	99.2 / 92.8	99.1 / 90.1	99.1 / 77.0	99.4 / 98.2
Mix	87.8 / 84.3	74.1 / 53.0	82.0 / 65.1	85.1 / 51.0	95.5 / 86.1	81.4 / 51.4	90.3 / 52.2	92.4 / 54.7	96.4 / 65.2	98.4 / 93.0
Avg	90.6 / 76.5	85.6 / 67.4	84.7 / 70.9	84.2 / 61.6	91.1 / 73.9	77.7 / 55.7	95.7 / 79.7	96.2 / 80.4	97.1 / 79.2	98.5 / 91.5

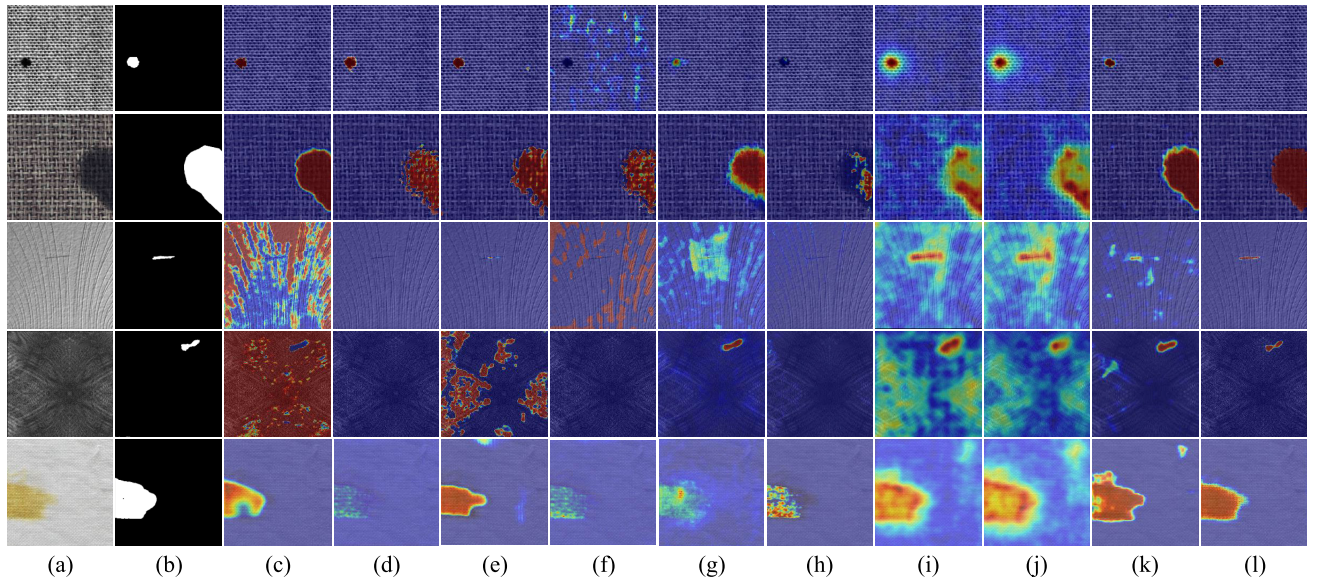


Fig. 11. Defect detection heatmaps of all selected methods. (a) Defective images. (b) Ground-truth defect masks. (c)–(l) Heatmaps of the Unet, SegNet, FSSNet, CGNet, DRÆM, MemSeg, RD, RD++, and Ours.

TABLE V
OPEN-SET AND CLOSED-SET CLASSIFICATION PERFORMANCE OF ALL SELECTED METHODS IN
ACCURACY (%) / F1 SCORE (%) / AUROC (%)

Method	Softmax	Openmax	CROSR	CSI	RPL	ARPL	CSSR	Ours
ZJU-Leaper	90.3 / 80.0 / 83.4	92.9 / 90.2 / 89.8	86.9 / 72.7 / 61.7	89.2 / 70.7 / 82.1	85.7 / 69.4 / 87.0	91.5 / 72.6 / 86.5	82.5 / 67.1 / 83.0	95.0 / 95.6 / 96.9
DAGM	97.6 / 76.4 / 59.8	96.4 / 90.1 / 90.2	87.5 / 88.9 / 77.0	93.8 / 71.5 / 73.5	86.4 / 82.2 / 79.4	97.8 / 92.9 / 68.4	97.5 / 75.8 / 71.0	99.2 / 95.6 / 97.8
MVTec	69.4 / 79.4 / 64.0	74.4 / 87.7 / 60.9	61.8 / 71.9 / 64.3	64.4 / 85.9 / 68.8	64.4 / 79.7 / 64.5	61.7 / 84.6 / 65.7	71.3 / 78.1 / 82.3	93.8 / 91.7 / 97.9
SZU-Fabric	87.1 / 77.8 / 67.0	81.5 / 84.1 / 66.2	82.8 / 63.0 / 60.1	85.8 / 63.6 / 81.1	78.9 / 50.5 / 72.5	88.4 / 62.5 / 70.4	86.4 / 60.4 / 78.7	89.8 / 86.0 / 94.8
Mix	81.3 / 74.6 / 69.1	83.1 / 86.1 / 77.1	84.7 / 77.4 / 77.7	78.5 / 72.8 / 79.2	84.6 / 81.7 / 83.2	80.1 / 84.2 / 82.1	79.6 / 84.1 / 84.9	91.8 / 88.8 / 98.4
Avg	85.1 / 77.6 / 68.7	85.7 / 87.6 / 76.8	80.7 / 74.8 / 68.2	82.3 / 72.9 / 76.9	80.0 / 72.7 / 77.3	83.9 / 79.4 / 74.6	83.5 / 73.1 / 80.0	93.9 / 91.4 / 97.2

1) *Ablation of Modules*: The DIS results of the proposed method on the Mix dataset are shown in Table VI. It can be seen that the performance is mediocre using only the

DIS module, but it improves significantly after adding the DGT module without the discriminator (w/o $\mathcal{D}$). Furthermore, after introducing the discriminator $\mathcal{D}$ to the DGT module, the

TABLE VI
INFLUENCE OF DIFFERENT MODULES ON DIS

DIS	DGT (w/o $\mathcal{D}$)	DGT	DFC	Image-level AUROC	Pixel-level AUROC
✓				67.5	81.2
✓	✓			88.1	98.1
✓		✓		90.2	98.1
✓			✓	77.1	89.7
✓	✓	✓	✓	94.8	98.4

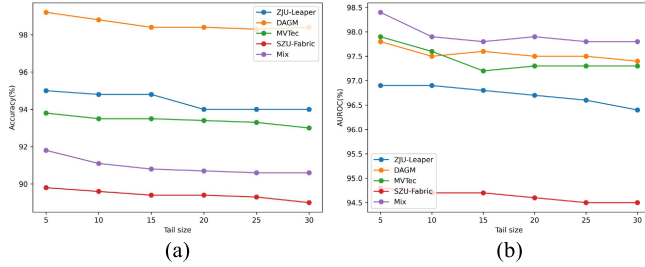


Fig. 12. Influence of different η values on performance. (a) Closed-set classification. (b) Open-set classification.

performance is even better, with an improvement of 22.7% and 16.9% in image- and pixel-level AUROC, respectively. Finally, the performance of DIS is 94.8% and 98.4% after the addition of all modules, which indicates the importance of each component for performance improvement.

2) *Effects of Hyperparameters:* The effect of the *tail size* hyperparameter η is also examined. Specifically, the proposed method is performed on the selected five datasets using the parameter varying from 5 to 30 with a step size of 5, and the closed-set and open-set classification performance is shown in Fig. 12. It can be seen that both the closed-set classification accuracy and the open-set recognition performance decrease gently as η increases, which may be attributed to that the proposed method could accurately extract the representative features of different defect types and consequently is relatively insensitive to the parameter η . As shown in Fig. 12, a smaller tail size is more effective in balancing closed-set classification accuracy and open-set detection performance, and thus, a tail size of 5 is selected for experiments.

F. Inference Speed

Experiments on the inference speed of all methods are conducted on the selected dataset, and the results are shown in Fig. 13, where each bar denotes the performance of different methods on the Mix datasets and the plotted line with dots represents their average performance on all datasets. Note that their performance is evaluated in terms of frames per second (FPS) and is averaged over five runs. It can be seen that the proposed method on the Mix dataset is the fastest among all selected methods, attaining an inference speed of 176.8 FPS. RD and RD++ perform well in defect detection, but their inference speed is very slow. The efficiency of UNet and SegNet seems to be good, yet their defect detection performance needs to be improved greatly. Among the comparative methods, DeSTSeg balances the detection performance and inference speed. However, compared to DeSTSeg, the

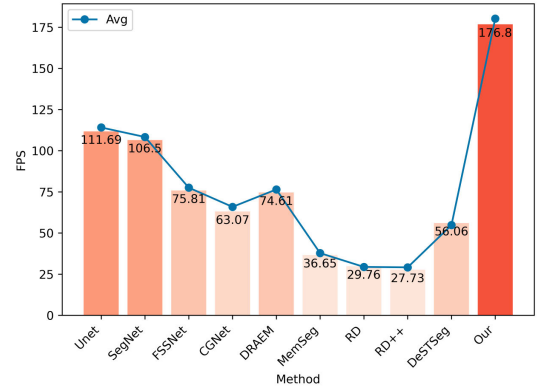


Fig. 13. Inference speed of all methods evaluated on the selected datasets.

inference speed of the proposed method is about three times faster and its detection performance is much better. For the average performance on all datasets, all methods have similar trends in inference speed, while the proposed method has the highest results, indicating its superior prospects in real-world industrial scenarios.

V. CONCLUSION

In this article, a novel open-set fabric defect detection method is proposed. The method introduces a DGT module to generate diverse defective samples with different background textures. With the defective samples generated by manipulating the random mask generator, a defect segmentation module is developed to identify and segment defects from fabric images. Moreover, a DFC module is designed, which further classifies identified defective images into known defect types or unknown types. Experiments conducted on several widely used benchmarks show that the proposed method is effective in DIS as well as DFC, achieving state-of-the-art performance in terms of different evaluation metrics. Despite the promising results achieved by the proposed method, there is still some work that deserves further investigation. On the one hand, fabrics manufactured in textile mills are dynamically changed according to the market demand, and thus, it is highly desired to design a universal fabric defect detection method to adapt different and unseen fabrics. On the other hand, it is expected to further learn from the detected unknown defects, which provides the ability for continual learning and lifelong learning.

REFERENCES

- [1] M. Patil, S. Verma, and J. Wakode, "A review on fabric defect detection techniques," *Int. Res. J. Eng. Technol.*, vol. 4, no. 9, pp. 131–136, 2017.
- [2] X. Xu, J. Chen, H. Zhang, and W. W. Y. Ng, "D4Net: De-deformation defect detection network for non-rigid products with large patterns," *Inf. Sci.*, vol. 547, pp. 763–776, Feb. 2021.
- [3] D. Xu, B. Lu, and B. Huang, "A multiscale convolutional registration network for defect inspection on periodic lace surfaces," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–9, 2022.
- [4] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, Jan. 2008.
- [5] V. Zavrtanik, M. Kristan, and D. Skocaj, "DRAEM—A discriminatively trained reconstruction embedding for surface anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 8310–8319.

- [6] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [7] R. Ren, T. Hung, and K. C. Tan, "A generic deep-learning-based approach for automated surface inspection," *IEEE Trans. Cybern.*, vol. 48, no. 3, pp. 929–940, Mar. 2018.
- [8] H. Yang, Y. Chen, K. Song, and Z. Yin, "Multiscale feature-clustering-based fully convolutional autoencoder for fast accurate visual inspection of texture surface defects," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 3, pp. 1450–1467, Jul. 2019.
- [9] Y. Zhao, K. Hao, H. He, X. Tang, and B. Wei, "A visual long-short-term memory based integrated CNN model for fabric defect image classification," *Neurocomputing*, vol. 380, pp. 259–270, Mar. 2020.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 2, pp. 84–90, Jun. 2012.
- [11] Z. Chen, J. Duan, L. Kang, and G. Qiu, "Supervised anomaly detection via conditional generative adversarial network and ensemble active learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7781–7798, Jun. 2022.
- [12] J. Liu, C. Wang, H. Su, B. Du, and D. Tao, "Multistage GAN for fabric defect detection," *IEEE Trans. Image Process.*, vol. 29, pp. 3388–3400, 2020.
- [13] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with cutout," 2017, *arXiv:1708.04552*.
- [14] S. Yun, D. Han, S. Chun, S. J. Oh, Y. Yoo, and J. Choe, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6022–6031.
- [15] V. Verma et al., "Manifold mixup: Better representations by interpolating hidden states," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6438–6447.
- [16] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 9664–9674.
- [17] M. Yang, P. Wu, and H. Feng, "MemSeg: A semi-supervised method for image surface defect detection using differences and commonalities," *Eng. Appl. Artif. Intell.*, vol. 119, Mar. 2023, Art. no. 105835.
- [18] T. D. Tien et al., "Revisiting reverse distillation for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 24511–24520.
- [19] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 2. Cambridge, MA, USA: MIT Press, 2014, pp. 2672–2680.
- [20] B. Lu, M. Zhang, and B. Huang, "Deep adversarial data augmentation for fabric defect classification with scarce defect data," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–13, 2022.
- [21] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, vol. 9351. Cham, Switzerland: Springer, 2015, pp. 234–241.
- [22] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.
- [23] X. Zhang, Z. Chen, Q. M. J. Wu, L. Cai, D. Lu, and X. Li, "Fast semantic segmentation for scene perception," *IEEE Trans. Ind. Informat.*, vol. 15, no. 2, pp. 1183–1192, Feb. 2019.
- [24] T. Wu, S. Tang, R. Zhang, J. Cao, and Y. Zhang, "CGNet: A light-weight context guided network for semantic segmentation," *IEEE Trans. Image Process.*, vol. 30, pp. 1169–1179, 2021.
- [25] H. Deng and X. Li, "Anomaly detection via reverse distillation from one-class embedding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 9737–9746.
- [26] X. Zhang, S. Li, X. Li, P. Huang, J. Shan, and T. Chen, "DeSTSeg: Segmentation guided denoising student-teacher for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 3914–3923.
- [27] C. S. C. Tsang, H. Y. T. Ngan, and G. K. H. Pang, "Fabric inspection based on the Elo rating method," *Pattern Recognit.*, vol. 51, pp. 378–394, Mar. 2016.
- [28] Y. Shi, J. Yang, and Z. Qi, "Unsupervised anomaly segmentation via deep feature reconstruction," *Neurocomputing*, vol. 424, pp. 9–22, Feb. 2021.
- [29] L. Shao, E. Zhang, Q. Ma, and M. Li, "Pixel-wise semisupervised fabric defect detection method combined with multitask mean teacher," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–11, 2022.
- [30] K. Song, H. Yang, and Z. Yin, "Anomaly composition and decomposition network for accurate visual inspection of texture defects," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–14, 2022.
- [31] A. Bendale and T. E. Boult, "Towards open set deep networks," in *Proc. IEEE Comput. Soc. Conf. Comput. Vision Pattern Recognit.*, Las Vegas, NV, USA, Jun. 2016, pp. 1563–1572.
- [32] R. Yoshihashi, W. Shao, R. Kawakami, S. You, M. Iida, and T. Naemura, "Classification-reconstruction learning for open-set recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4016–4025.
- [33] G. Chen, P. Peng, X. Wang, and Y. Tian, "Adversarial reciprocal points learning for open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 8065–8081, Nov. 2022.
- [34] H. Huang, Y. Wang, Q. Hu, and M.-M. Cheng, "Class-specific semantic reconstruction for open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4214–4228, Apr. 2023.
- [35] Y. Choi, M. Choi, M. Kim, J. Ha, S. Kim, and J. Choo, "StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8789–8797.
- [36] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis.*, vol. 11211, Sep. 2018, pp. 3–19.
- [37] W. J. Scheirer, A. Rocha, R. J. Micheals, and T. E. Boult, "Meta-recognition: The theory and practice of recognition score analysis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 8, pp. 1689–1695, Aug. 2011.
- [38] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4182–4191.
- [39] J. Mukhoti, A. Kirsch, J. van Amersfoort, P. H. Torr, and Y. Gal, "Deep deterministic uncertainty: A new simple baseline," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24384–24394.
- [40] J. Tack, S. Mo, J. Jeong, and J. Shin, "CSI: Novelty detection via contrastive learning on distributionally shifted instances," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 11839–11852.
- [41] G. Chen et al., "Learning open set network with discriminative reciprocal points," in *Proc. 16th Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 507–522.



Can Gao (Member, IEEE) received the Ph.D. degree from Tongji University, Shanghai, China, in 2013.

From 2010 to 2011, he was a Visiting Scholar with the University of Alberta, Edmonton, AB, Canada. From 2015 to 2018, he was a Research Associate, a Post-Doctoral Fellow, and a Research Fellow with The Hong Kong Polytechnic University, Hong Kong. He is currently an Associate Professor with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China. He has authored or co-authored more than 60 academic articles. His main research interests include semisupervised learning, anomaly detection, and soft computing.



Xiuqian Chen received the B.E. degree from Guangxi University, Nanning, China, in 2022. He is currently pursuing the M.E. degree with the School of College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China.

His research interests include anomaly detection and computer vision.



Jinbao Wang (Member, IEEE) received the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2019.

He is currently an Assistant Professor at the National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen, China. His main research interests include computer vision and machine learning, with a long-term focus on image anomaly detection, image generation, and fast retrieval.



Jie Zhou (Member, IEEE) received the Ph.D. degree from Tongji University, Shanghai, China, in 2011.

From 2010 to 2011, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB, Canada. From 2017 to 2018, he was an Associate Researcher at The Hong Kong Polytechnic University, Hong Kong. He is currently an Associate Professor with the National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen, China. His main

research interests include pattern recognition, computer vision, and intelligent systems.



Linlin Shen (Senior Member, IEEE) received the Ph.D. degree from the University of Nottingham, Nottingham, U.K., in 2005.

He is currently a Pengcheng Scholar Distinguished Professor with the School of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China, and an Honorary Professor with the School of Computer Science, University of Nottingham, Nottingham, U.K. He is also the Director of the Computer Vision Institute, AI Research Center for Medical Image Analysis and Diagnosis,

Shenzhen University, and China–U.K. Joint Research Laboratory, Shenzhen, for Visual Information Processing. His research interests include deep learning, facial recognition, analysis/synthesis, and medical image processing.

Dr. Shen is the Co-Editor-in-Chief of *IET Journal of Cognitive Computation and Systems* and an Associate Editor of *Expert Systems with Applications*. He is listed as the Most Cited Chinese Researchers by Elsevier and listed in a ranking of the Top 2% Scientists in the World by Stanford University.